

CS 6349.001 – Network Security

Fall 2025 Programming Project

Due Date: TBD

Project Overview

In this project, you will build an **Anonymous Secure Chat System**. The system consists of a central **Relay Server** and multiple **Clients**. Clients do not communicate directly with each other; instead, all communication is relayed through the server.

At the start, Clients and the Relay only know each other's network addresses and long-term identifiers. The Relay cannot be trusted with message contents, and Clients should not reveal their real identities. Each Client therefore registers with the Relay under a **pseudonym** and relies on the Relay only for message delivery.

Your task is to design a protocol that allows Clients to establish secure sessions through the Relay while ensuring that:

- The Relay can forward messages correctly but cannot read them.
- Clients can trust that messages come from the correct source.
- Communication remains private, tamper-evident, and resistant to replay.
- Past conversations stay protected even if long-term keys are compromised later.
- External entities should not be able to determine which Clients are communicating with each other (end-point identity hiding, to the extent possible).

This project will give you practical experience in designing secure communication protocols and implementing them using network sockets under realistic constraints.

Supported Functionalities

- A Client should be able to register with the Relay under a **pseudonymous ID**.
- A Client should be able to initiate a chat session with another Client through the Relay.
- All communication between Clients must pass through the Relay, but the Relay must not learn the contents of the messages.
- A Client should be able to send and receive text messages securely once a session is established.
- Communication should be designed to provide **end-point identity hiding**:
 - Clients will know the pseudonyms of the peers they are communicating with.
 - External entities (e.g., eavesdroppers) should not easily determine which Clients are talking to each other.

Security Requirements

Your design must ensure the following:

- **Authentication**
 - Clients must verify that they are talking to the correct Relay.
 - The Relay must verify that each message actually comes from the registered Client.
- **Confidentiality**
 - The Relay should not be able to read the contents of any message it forwards.
 - Encryption must be designed using **keyed-hash based constructions**, not standard symmetric encryption algorithms.
- **Integrity**
 - Clients should be able to detect if a message was tampered with in transit.
- **Replay Protection**
 - Old messages should not be accepted as new ones.
- **Forward Secrecy**
 - Past communications must remain secure even if long-term keys are compromised in the future.
- **End-Point Identity Hiding**
 - While Clients know each other's pseudonyms, external entities (e.g., an eavesdropper) should not easily determine which Clients are communicating with each other.

What to Submit

- **Team Work:** You may work individually or in teams of up to two members.
- **Design Report**
 - Must describe your proposed chat protocol.
 - Clearly outline the protocol phases (registration, authentication, session setup, message exchange).
 - Explain how your design ensures confidentiality, integrity, anonymity, and forward secrecy.
 - Identify a threat model, including potential attacks and how they are handled.
- **Final Report**
 - Should summarize your final protocol and implementation.
 - Must include a detailed security analysis of your design.
 - Contributions of each team member must be clearly documented.

Technical Notes

- You need to use a programming language from {C/C++, Java, Python, Golang}.
- Keys may be generated using tools such as ssh-keygen or online RSA/DH key generators.
- You may assume that each party already has the necessary long-term RSA key pairs.

- All encryption in your design must be constructed using keyed-hash functions rather than standard symmetric ciphers.
- You are not supposed to use any secure transport layer service implementations. You may use suitable crypto libraries only for 1) basic functions of public-key cryptosystems (i.e., sign, verify, encrypt, decrypt) and 2) a chosen hash function (e.g., sha256) for use in your design of keyed-hash

References:

1. <https://www.geeksforgeeks.org/java/introducing-threads-socket-programming-java/>
2. <https://www.geeksforgeeks.org/cpp/socket-programming-in-cpp/>